Justification of Coding

1. Architecture and class design

Firstly, my game is composed of several different types of entities and solids - all of which inherit from *pygame.sprite.Sprite*. For example, the *Player* and *Enemy* classes - they share common attributes such as a rect hitbox, sprite, and position. This allows for the specific classes to be extended into their particular use-cases, such as how the player has wax and spell casting and the enemy has pathfinding and attack cooldowns. This also applies to the *Wall* and *Candle* objects, which are grouped into a *solids_group* allowing for easy collision between all entities and solids. Thus, walls, doors and candles become collision objects without unique collision code. Secondly, Spell and Enemy type definitions are kept in data tables (*ENEMY_DATA* & *SPELL_DATA*) because they are static content with no behaviour. These are accessed by their respective *Enemy / EnemyAttack* and *Flame* classes, which allows enemies and flames to take on any amount of shapes or sizes so to speak without hard-coding new variants into the Classes themselves.

2. Storage strategy

Data in Candlelight is split between two categories - static content, which remains hardcoded and rests in dictionaries or variables defined in *main.py*, and dynamic player-specific content which is stored in an SQLite database, *candlelight.db*. I made this split because I wanted to implement persistent data for the player, tracking things such as lifetime plays (*total_runs*) and highscores which can be stored even when the player exits out of the game, not in some temporary count variable which is defined as zero whenever the player starts the game. As such, to efficiently store the data I chose to use the *sqlite3* module which is native to python, instead of using something like writing to a text file as I have done in previous projects, which is much slower and requires you to write your own compiler/database-reader. Many variables were kept separate from this dynamic content because they are simply static and may be defined at launch. This is more efficient than purposelessly drawing from an SQL table every time I need to access variables like *FPS* or colours. Additionally, static content was split between individual variables and tuples for simple things like *BROWN* (150, 80, 0), and dictionaries used to store more complex similar data sets such as *ENEMY_DATA* which allows me to efficiently create new enemy types on the fly, and store as many types as I want.

3. Choice of algorithms

The two major algorithms I implemented in Candlelight were A* pathfinding for my *Enemy* class, and Cellular Automata for my cave/dungeon generation. I chose to develop A* pathfinding because of the complexity of my dungeon levels - if I had opted to remain with the simple follow-the-player algorithm I put together in early development, enemies would far too often get

stuck on walls or be unable to navigate through even minutely complex passages. A* was very effective for this approach because it uses a heuristic function to estimate remaining distance, which given my game operates on a tile grid system, is very efficient given it can quickly narrow down on the few tiles which will have the shortest path. I could have used an algorithm which searches every possible path, but since I have a limited amount of tiles and the enemies don't need to be as fast as mathematically possible, A* was effective because it only calculates promising paths. Since my enemies only move in 4 possible directions, I implemented Manhattan distance as my heuristic of choice. A more dynamic, omnidirectional enemy would require a different kind of heuristic. Additionally as my second complex algorithm, I implemented cellular automata dungeon generation instead of stitching together several premade rooms. I chose this approach because I wanted to challenge myself in learning to build more complex algorithms, and to give my game a major element which is fresh on every play - two rooms will rarely ever be the same, which will help to prevent the game feeling stale after a few plays. Cellular automata has been so effective for the job of generating organic cave structures in my game because it uses an incredibly simple (relative to others) algorithm to generate rooms which are unique but still follow some guiding principles and rules to allow the rooms to be actually playable. The algorithm first generates a random noise map, turning about 45% of tiles into a Wall tile, and then applies a smoothing algorithm over four iterations. The *smooth_map* function counts each tile's 8 neighbours. If more than 4 are walls, the tile becomes a wall. If less than 4 are walls, the tile becomes a floor block. If exactly 4 are walls, the tile's state remains unchanged. Additionally I added some simple rules for post-processing - the tiles around the player's spawn MUST be floor to avoid a softlock, a solid border is added around the edges of the map, and door tiles can only be placed on top of floor tiles. As a limitation, cellular automata is too simple an algorithm to guarantee all "caves" are connected - some parts of the map may remain isolated from the player's field of play, though I implemented an *find_reachable_tiles()* function to ensure all necessary objects are within possible reach of the player.

4. Modular structure

Candlelight is innately structured in a modular form. Each process belongs to a subroutine or function, such as the *is_walkable()* function, which returns True or False to a tile being 'walkable' instead of this process being defined within some other composite function. This allows candlelight's menu/game loops to be incredibly readable, and also allows me as the developer to understand where any problem may be coming from and to understand the general structure of the code without getting lost in bloated loops or functions. Some functions were also defined and then combined to form concise composite functions, for example, the *draw_hud()* function is very simply composed of just *draw_queue()* and *draw_wax()*. The two minor-functions perform the singular process of drawing the spell queue and the wax bar for the player, and the *draw_hud()* composite delivers these in the singular process of drawing the HUD, instead of individually defining the two tasks of drawing the queue and the wax. Thus, the main loop of candlelight is readable, descriptive and succinct.

5. Performance decisions

I made several performance and optimisation decisions in the development of candlelight to ensure the maximisation of playability for users. For example, I limited the recalculation of enemy paths to once every 30 frames or half-seconds. If my enemy recalculated it's A* path more frequently, for example every frame, it would lead to a significant decrease in performance given the high load of completing this process. However, I had to balance performance with my enemies actually performing the task they were built for, chasing the player. If their paths were recalculated too infrequently, such as every 5 seconds, they would very easily lose sight of the player and would make for a confusing, irritating and boring player experience. I verified my performance concerns through volume testing, measuring the framerate of Candlelight with progressively more concurrent entities (and therefore more path-recalculations per second), as seen in my test data document. For example, I found that with 2-20 enemies my game consistently ran at speeds far higher than necessary, but when exceeding 100-300 enemies my game slowed down considerably so that anything further may affect the user experience.

6. Security and portability

Since Candlelight is a fully offline, single player, local game, standard web concerns such as authentication, XSS or transit encryption don't need to be considered. What does need to be considered however, are input validation, portability and data protection. In terms of input validation, the only runtime input from the user is the keyboard, which python already normalises into discrete events - no free text or integer is ever accepted from the user. Additionally, functions like *is_walkable()* use bounds guards to simply return False on out-of-index inputs, avoiding something like an IndexError crash. I also used parameterised SQL queries using *?* placeholders in *record_run()*, preventing malformed data from breaking the query (and technically avoiding SQL injection, though that shouldn't be an issue single-player). In terms of data protection, no sensitive data about the user is ever stored, so data leaks are not an issue. Additionally, *candlelight.db* is left unencrypted, since the only person with access is the user themselves there is no need to put in the extra effort for encryption which is industry standard across offline games. In terms of portability, I didn't use any hardcoded URLs or absolute paths, as for example *DB_NAME = candlelight.db* is relative and should work on any machine. Additionally, player profile data will be available for all users, as if *candlelight.db* is not present on startup it will simply be created. Additionally, *requirements.txt* makes it easy to understand and install the required dependencies, whilst modules such as *sqlite3* are simply native to python. Finally, all graphics are drawn programatically, meaning there is no room for error where the program may be missing assets.

7. Deliberate limitations

The *brute* enemy in Candlelight is intentionally limited in its movement options - given Candlelight operates on a 32px tile system and the brute has a width of 40px, it is unable to navigate through single-file or tighter passages, and has a harder time getting around corners and generally pathfinding to the player. This limitation was made intentional given the brute's high damage potential, adding a level of depth to the game where the player can 'outsmart' the larger enemies in the game.

8. Deviations from design document

Several deviations from the design document were made in order to maximise the playability of Candlelight and manage its completion in accordance with deadlines. Firstly, features which were dropped include the upgrade shop, spell customisation and multiple player-characters. Importantly, these features belonged to the SHOULD or COULD sections of my design document, and all MUST have features were completed and implemented. However, in my development I did exceed or work around some other expectations, for example using a wax bar instead of the concept candle-flame UI, using a SPELL_DATA table instead of building a spell class, and importantly building A* instead of simpler follow-the-player and cellular automata instead of hand-made dungeon rooms. I made these changes because features such as an upgrade shop and spell customisation were not deemed necessary to the player experience, whilst I believed that A* and cellular automata would greatly improve the playability of Candlelight and provide more to the player with less from the developer. As such I kept in line with my scope-creep prediction from part 1, managing the necessary features and nice-to-have features in my development of this major project.